

Predictive Coding Predictors for Joint-Embedding World Models

Progress report and request for feedback | Yash Nagraj | August 2026

Where this stands. The project began as a proposal to train the predictor of an I-JEPA-style joint-embedding world model with predictive coding (PC) instead of backpropagation (BP). The original motivation was that PC might supply a better gradient signal. I now think that motivation was wrong, and I have replaced it with a different one. This report covers what I built, what I measured, what changed my mind, and three questions I would like your opinion on. Everything numerical below is from code I wrote and ran; nothing is quoted from a paper.

1. The reframing: PC does not give better gradients

Millidge, Tschantz and Buckley (2020) show that PC approximates BP along arbitrary computation graphs. So “PC gives a better gradient than BP” is close to self-defeating: to the extent PC works, it works *because* it recovers the BP gradient. I verified this myself rather than take it on faith — I implemented PC over an explicit computation graph in JAX and compared it against autodiff on three topologies (a plain chain, a residual network with folded skips, and one with explicit skips). The PC and BP weight gradients agree to **cosine similarity 1.000000** on all three. On a single branch the PC gradient is **-11.370010** against BP’s **-11.370533**.

Two things fell out of that exercise that I did not expect, and that I think are the more useful results:

- **The equivalence is load-bearing on one assumption.** Dropping the fixed-prediction assumption changes that same gradient from **-11.370010** to **-3.413946** — a factor of three, not a rounding error. The agreement with BP is not robust; it is contingent.
- **Error propagates as a wavefront whose speed is set by graph topology.** The number of inference steps before a node first receives a non-zero error equals its *shortest path to the output*, exactly. For the chain I get arrival times [5,4,3,2,1,0]; adding explicit skips gives [3,2,2,1,1,0]. The consequence is a floor on the inference budget: $T \geq L$ for explicit skips versus $T \geq 2L$ for folded ones. Skip connections are not a detail in PC — they halve the required relaxation.

So if PC is worth pursuing here, it is not for gradient quality. The question I actually care about is whether **the relaxation process produces different (and for patch prediction, better) latent representations** than a forward pass does. That is a question about representations, not about credit assignment, and as far as I can tell it is open.

2. Scaling the architecture: reproducing μ PC

To get to anything JEPA-shaped I need depth, residual connections, and autodiff. I therefore worked through μ PC (Innocenti et al.), which reparameterises PC networks in a depth- μ P style so that deep residual PCNs train at all. I reproduced their setting from scratch: a **30-layer fully-connected ResNet, width 128, on MNIST**, using the `jpc` library, with a standard-parameterisation (SP) control at identical depth, weights and learning rates.

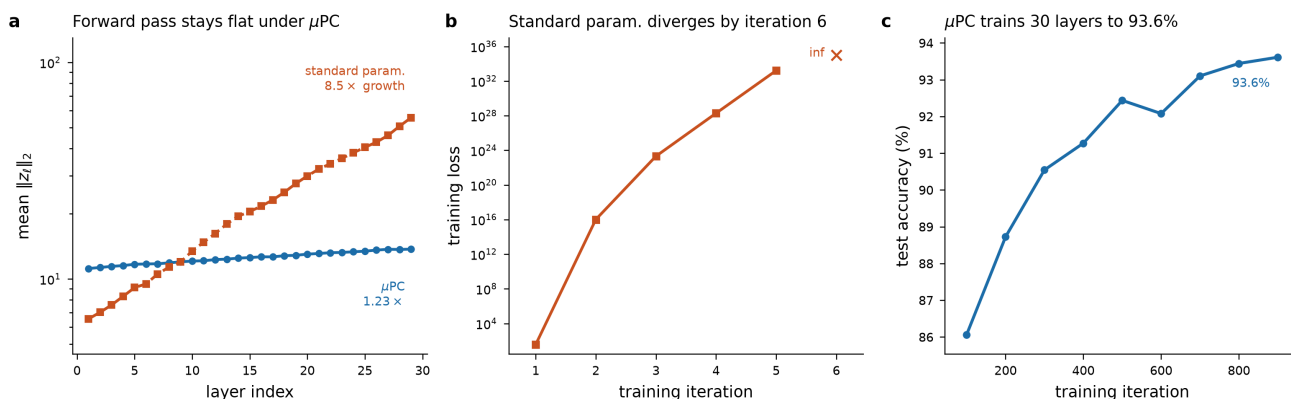


Figure 1. All three panels are from my own runs at depth 30, width 128, batch 64, activity lr 0.5, weight lr 0.1 (Adam), seed 4329. (a) Mean activity norm per layer after a forward pass at initialisation; log scale; the output layer is omitted since its scaling differs by construction. (b) SP training loss per iteration until it becomes non-finite. (c) μ PC test accuracy over 900 iterations (≈ 1 epoch), full 10k test set.

The control is unambiguous. Under SP the forward pass grows $8.5\times$ from the first to the last hidden layer at initialisation ($6.53 \rightarrow 55.55$), and training diverges within six iterations — the loss goes $36.5, 1.0\times 10^{16}, 2.1\times 10^{23}, 1.9\times 10^{28}, 1.7\times 10^{33}, \text{inf.}$ Under μPC the same forward pass grows $1.23\times$ ($11.17 \rightarrow 13.73$) and the network trains smoothly to **93.61%** test accuracy in a single epoch. This is the paper’s headline claim, reproduced independently.

3. The result that actually surprised me

I instrumented the training loop to record the PC energy $F(z)$ at every one of the $T = 30$ inference steps, so I could check whether the relaxation was doing any work. It is doing almost none:

training iteration	$F(z)$ before inference	$F(z)$ after $T=30$ steps	relative drop
100	0.1618	0.1610	0.49%
200	0.1228	0.1222	0.49%
300	0.0978	0.0973	0.51%
400	0.0935	0.0930	0.53%

Thirty steps of gradient descent on the activities move the energy by half a percent, and the figure is stable across training. In other words, under μPC **the feedforward initialisation already lands essentially at the inference fixed point**. That is a coherent reading of why μPC works — the paper shows the inference landscape becomes badly ill-conditioned with depth, and the reparameterisation appears to sidestep that by making relaxation nearly unnecessary rather than by making it converge faster.

But it is awkward for my project. If inference is a 0.5% correction to a forward pass, then the latents PC produces are, to a very good approximation, the latents a forward pass produces — and the “richer representations” hypothesis from §1 has little room to be true in this regime. Either the effect I am looking for lives somewhere other than the discriminative-MNIST setting, or it needs a network where relaxation genuinely moves the activities. **This is the single point I would most like a second opinion on.**

I should flag a limitation honestly: the depth sweep across $\{8, 16, 30, 64\}$ that would show how this scales is written but has not finished running, and my current numbers are single-seed. I am not claiming statistical strength for the 0.5% figure — only that it is consistent across four widely-separated logging points in one run.

4. A direction I have just been pointed to: MPC / NeuroSSL

Prof. Alexander Ororbia, whom I contacted about this project, pointed me to **Meta-Representational Predictive Coding** (Ororbia, Friston and Rao, arXiv:2503.21796), which he describes as the first encoder-centric or joint-embedding form of predictive coding. From its abstract, MPC avoids learning a generative model of raw pixels by predicting *representations* of the input across parallel streams, giving an encoder-only learning and inference scheme, with representational dynamics driven by active sensory glimpsing. He also noted that MPC ties the feedback weights to the forward ones — the usual PC simplification — and that there is no reason a structure like mine could not instead **learn the error weights**, as in neural generative coding (NGC).

I have not read this paper yet, so I am not going to characterise its results here; it is the next thing I read, and I am flagging it because it clearly changes the shape of the project. Two implications I can already see:

- It suggests the joint-embedding + PC combination is viable in a form much closer to what I proposed than anything in the μPC line, which is purely discriminative.
- Learned (untied) error weights are a concrete axis I had not considered at all. Everything I have built so far assumes tied feedback, which is exactly the assumption that made my BP-equivalence check come out at cosine 1.0. Untying it would break that equivalence — which, given §1, might be the point rather than a problem.

5. What I have running, and what is next

component	status
PC over arbitrary computation graphs (JAX), 3 topologies	done, verified against autodiff
Error-wavefront / inference-budget analysis	done; $T \geq L$ vs $T \geq 2L$ result is exact
μPC 30-layer ResNet on MNIST + SP control	done, reproduces the paper
Per-step inference-energy instrumentation	done; gives the 0.5% result above

Depth sweep {8, 16, 30, 64}, multi-seed	written, not yet run
Swap MNIST classification for patch prediction	next
Read MPC; evaluate untied / learned error weights	next

The immediate plan is to keep the μ PC-parameterised residual encoder but replace the supervised objective with a JEPA-style one — predict the representation of a masked patch from the representation of the context — and then re-run the same energy instrumentation. If relaxation still only moves the energy half a percent in that setting, the representational hypothesis is in trouble and I would rather find that out in the next two weeks than in six months.

6. Questions I would like your opinion on

1. Is the “richer latents” framing the right question, or am I chasing something that the 0.5% energy result has already answered negatively? If the forward pass lands at the fixed point, is there a regime — different objective, weaker skips, smaller inference learning rate, untied weights — where relaxation would genuinely reshape the representation, or is that simply not what PC buys you?

2. How would you actually measure “richer”? This is my biggest methodological gap. Candidates I have considered: linear-probe accuracy on held-out tasks, effective rank of the representation matrix, resistance to representational collapse, downstream patch-prediction error. None feels decisive, and the μ PC paper explicitly does not study representation quality at all.

3. Are there directions in this space I should know about? The MPC pointer above reoriented the project in a single email; I would rather discover the rest of that literature now than after building the wrong thing. In particular, is anyone applying NGC-style learned error weights in a joint-embedding setting, and is there work on PC for masked prediction specifically?

And the blunt version: **are these efforts pointed in a sensible direction?** I would rather be redirected early. All code, notebooks and figures are available if any of the above is worth looking at directly.

References: Millidge, Tschantz & Buckley, *Predictive Coding Approximates Backprop along Arbitrary Computation Graphs* (2020). Innocenti et al., *μ PC: Scaling Predictive Coding to 100+ Layer Networks*. Ororbia, Friston & Rao, *Meta-Representational Predictive Coding*, arXiv:2503.21796. Assran et al., *I-JEPA* (2023). Implementation uses the jpc library (Buckley lab).